The output of the program indicates that you need to specify an order. You do that with arguments.

Program arguments

Any Node.js program run on the command line has access to the command-line arguments passed to it via the `process`. The first command-line argument is available at `process.argv[2]`. Suppose the program is invoked as follows:

```
> coffee 2.4.coffee 'Cappuccino'
```

Here, the program `process.argv[2]` is `'Cappuccino'`. What happened to `argv[0]` and `argv[1]`? They're reserved for other properties. The `process.argv[0]` is the runtime (in this case, `coffee`) and the `process.argv[1]` is the filesystem path for the program executed (in this case, the full path to the 2.4.coffee file).

The filesystem module

The other essential task in a Node.js program is to read files. For example, suppose your command-line barista program needs to read the house roast from a file before serving a coffee. To do this in Node, you'll require the filesystem module:

```
fs = require 'fs'
```

At this point you don't need to know much about how the module system works. It's covered in depth in chapter 12. Back to the program, though; in the next listing you see a full implementation of the command-line barista program.

Listing 2.4. A command-line barista